VG151-25FA BigRC2 - Algorithm

Homework 4 Ex. 2 - Recursion, basic arithmetic

Problem: Write a recursive algorithm to calculate the GCD of two integers.
Pseudocode:

Algorithm 1: Recursive GCD Algorithm (Euclidean Algorithm)

```
Input  : Two integers a and b
Output: GCD of a and b

Function GCD(a, b):
    if b = 0 then
        return a;
    end if
    return GCD(b, a mod b);
end
```

Time Complexity: $\mathcal{O}(\log(\min(a, b)))$
Explanation: The Euclidean algorithm repeatedly replaces the larger number with the remainder of the division. The number of recursive calls is proportional to the number of digits in the smaller number, which is logarithmic in the value.

Homework 5 Ex. 1 - Array

Problem: Find all prime numbers less than n and store them in an array.
Pseudocode:

Algorithm 2: Sieve of Eratosthenes

```
Input  : Integer n
Output: Array of all primes less than n

Function FindPrimes(n):
    Create boolean array isPrime[2..n − 1] initialized to true;
    Create empty array primes for storing results;
    for i ← 2 to √n do
        if isPrime[i] is true then
            j ← i^2;
            while j < n do
                isPrime[j] ← false;
                j ← j + i;
            end while
        end if
    end for
    for i ← 2 to n − 1 do
        if isPrime[i] is true then
            Append i to primes;
        end if
    end for
    return primes;
end
```

Time Complexity: $\mathcal{O}(n \log \log n)$
Explanation: The Sieve of Eratosthenes marks multiples of each prime. The outer loop runs $\mathcal{O}(\sqrt{n})$ times, and the inner loop operations sum to $\mathcal{O}(n \log \log n)$ based on the harmonic series of primes.

Homework 5 Ex. 4 - Loops, standard library, mathematical functions

Problem: Use the bisection method to find when a projectile touches the ground. The trajectory equation is:

$$y = h_0 + \tan(\theta_0)x - \frac{g}{2v_0^2\cos^2(\theta_0)}x^2$$

Pseudocode:

Algorithm 3: Bisection Method for Projectile Landing

Input : v_0, θ_0, h_0, a, b (initial interval $[a, b]$)
Output: Position x where projectile touches ground

```
Function Trajectory(x, v_0, θ_0, h_0):
    g ← 9.81;
    θ_rad ← θ_0 × π/180;
    y ← h_0 + tan(θ_rad) × x − g/(2 v_0^2 cos^2(θ_rad)) × x^2;
    return y;
end
Function Bisection(v_0, θ_0, h_0, a, b):
    while (b − a) > FLT_EPSILON do
        mid ← (a + b)/2;
        f_mid ← Trajectory(mid, v_0, θ_0, h_0);
        f_a ← Trajectory(a, v_0, θ_0, h_0);
        if f_mid × f_a < 0 then
            b ← mid;
        end if
        else
            a ← mid;
        end if
    end while
    return (a + b)/2;
end
```

Time Complexity: $\mathcal{O}(\log((b-a)/\epsilon))$ where ϵ is FLT_EPSILON
Explanation: The bisection method halves the interval in each iteration. The number of iterations needed is $\log_2(\frac{b-a}{\epsilon})$.

Homework 5 Ex. 6 - Low level C programming

Problem: Implement efficient bit counting using divide and conquer approach with bitwise operators. Assuming 64-bit integer

Pseudocode:

Algorithm 4: Count Leading Zeros (Divide and Conquer)

```
Input  : Unsigned integer x
Output: Number of leading zeros

Function CountLeadingZeros(x):
    if x = 0 then
        return 64
    end if
    n ← 0;
    if (x&0xFFFFFFFF00000000) = 0 then
        n ← n + 32;
        x ← x ≪ 32;
    end if
    if (x&0xFFFF000000000000) = 0 then
        n ← n + 16;
        x ← x ≪ 16;
    end if
    if (x&0xFF00000000000000) = 0 then
        n ← n + 8;
        x ← x ≪ 8;
    end if
    if (x&0xF000000000000000) = 0 then
        n ← n + 4;
        x ← x ≪ 4;
    end if
    if (x&0xC000000000000000) = 0 then
        n ← n + 2;
        x ← x ≪ 2;
    end if
    if (x&0x8000000000000000) = 0 then
        n ← n + 1;
    end if
    return n;
end

Function BitPosition(x):
    return 64− CountLeadingZeros(x);
end
```

Time Complexity: $\mathcal{O}(\log w)$ where w is the word size (number of bits)

Explanation: Instead of iterating through each bit ($\mathcal{O}(w)$), the divide and conquer approach checks half the bits at each step, reducing complexity to $\mathcal{O}(\log w)$. For a 64-bit integer, this means at most 6 comparisons instead of 64 iterations.

e2r Ex. 4 - Tower of Hanoi

Problem: Solve the Tower of Hanoi puzzle: move n disks from Peg 1 to Peg 2 using Peg 3 as temporary, with the constraints that (i) only one disk is moved at a time, and (ii) a larger disk is never placed on top of a smaller disk.

Pseudocode:

Algorithm 5: Tower of Hanoi (Recursive Solution)

Input : Number of disks n, source peg *from*, destination peg *to*, auxiliary peg *aux*
Output: Sequence of moves and total move count

```
Function Hanoi(n, from, to, aux):
    if n = 1 then
        MoveDisk(1, from, to);
        return;
    end if
    Hanoi(n − 1, from, aux, to);
    MoveDisk(n, from, to);
    Hanoi(n − 1, aux, to, from);
end
```

Time Complexity: $\mathcal{O}(2^n)$
Explanation: The recurrence relation is $T(n) = 2T(n-1) + 1$ with $T(1) = 1$. This solves to $T(n) = 2^n - 1$, which is exponential. Each disk requires moving all smaller disks twice (once to auxiliary, once to destination), plus one move for itself.

Recursive - Quick Sort

Problem: Sort an array using the Quick Sort algorithm with divide-and-conquer strategy.

Pseudocode:

Algorithm 6: Quick Sort (Recursive)

Input : Array A, left index *low*, right index *high*
Output: Sorted array A

```
Function QuickSort(A, low, high):
    if low < high then
        pivotIndex ← Partition(A, low, high);
        QuickSort(A, low, pivotIndex − 1);
        QuickSort(A, pivotIndex + 1, high);
    end if
end

Function Partition(A, low, high):
    pivot ← A[high];
    i ← low − 1;
    for j ← low to high − 1 do
        if A[j] ≤ pivot then
            i ← i + 1;
            Swap A[i] and A[j];
        end if
    end for
    Swap A[i + 1] and A[high];
    return i + 1;
end
```

Time Complexity: Average: $\mathcal{O}(n \log n)$, Worst: $\mathcal{O}(n^2)$
Explanation: Average case occurs when partitions are balanced, giving $T(n) = 2T(n/2) + \mathcal{O}(n)$. Worst case occurs when pivot is always smallest/largest element (already sorted array), giving $T(n) = T(n-1) + \mathcal{O}(n) = \mathcal{O}(n^2)$.

Recursive - Merge Sort

Problem: Sort an array using the Merge Sort algorithm with divide-and-conquer strategy.
Pseudocode:

Algorithm 7: Merge Sort (Recursive)

```
Input  : Array A, left index left, right index right
Output: Sorted array A

Function MergeSort(A, left, right):
    if left < right then
        mid ← ⌊(left + right)/2⌋;
        MergeSort(A, left, mid);
        MergeSort(A, mid + 1, right);
        Merge(A, left, mid, right);
    end if
end

Function Merge(A, left, mid, right):
    n_1 ← mid − left + 1;
    n_2 ← right − mid;
    Create temporary arrays L[0..n_1 − 1] and R[0..n_2 − 1];
    for i ← 0 to n_1 − 1 do
        L[i] ← A[left + i];
    end for
    for j ← 0 to n_2 − 1 do
        R[j] ← A[mid + 1 + j];
    end for
    i ← 0, j ← 0, k ← left;
    while i < n_1 and j < n_2 do
        if L[i] ≤ R[j] then
            A[k] ← L[i];
            i ← i + 1;
        end if
        else
            A[k] ← R[j];
            j ← j + 1;
        end if
        k ← k + 1;
    end while
    while i < n_1 do
        A[k] ← L[i];
        i ← i + 1, k ← k + 1;
    end while
    while j < n_2 do
        A[k] ← R[j];
        j ← j + 1, k ← k + 1;
    end while
end
```

Time Complexity: $\mathcal{O}(n \log n)$ for all cases
Explanation: The algorithm always divides the array into two halves, giving $T(n) = 2T(n/2) + \mathcal{O}(n)$ where the merge operation takes linear time. This recurrence solves to $\mathcal{O}(n \log n)$ regardless of input.

Recursive - Pre-order Traversal

Problem: Traverse a binary tree in pre-order (Root-Left-Right).
Pseudocode:

Algorithm 8: Pre-order Traversal (Recursive)

Input : Root node *node* of binary tree
Output: Pre-order sequence of nodes

```
Function PreOrder(node):
    if node = NULL then
        return;
    end if
    Visit node (e.g., print node.data);
    PreOrder(node.left);
    PreOrder(node.right);
end
```

Time Complexity: $\mathcal{O}(n)$ where n is the number of nodes
Explanation: Each node is visited exactly once. The function makes two recursive calls per node but processes each node only once, resulting in linear time complexity.

Recursive - In-order Traversal

Problem: Traverse a binary tree in in-order (Left-Root-Right).
Pseudocode:

Algorithm 9: In-order Traversal (Recursive)

Input : Root node *node* of binary tree
Output: In-order sequence of nodes

```
Function InOrder(node):
    if node = NULL then
        return;
    end if
    InOrder(node.left);
    Visit node (e.g., print node.data);
    InOrder(node.right);
end
```

Time Complexity: $\mathcal{O}(n)$ where n is the number of nodes
Explanation: Each node is visited exactly once. For binary search trees, in-order traversal produces nodes in sorted order. The time complexity is linear as every node is processed once.

Recursive - Post-order Traversal

Problem: Traverse a binary tree in post-order (Left-Right-Root).
Pseudocode:

Algorithm 10: Post-order Traversal (Recursive)

Input : Root node *node* of binary tree
Output: Post-order sequence of nodes

```
Function PostOrder(node):
    if node = NULL then
        return;
    end if
    PostOrder(node.left);
    PostOrder(node.right);
    Visit node (e.g., print node.data);
end
```

Time Complexity: $\mathcal{O}(n)$ where n is the number of nodes
Explanation: Each node is visited exactly once. Post-order traversal is useful for deleting trees or evaluating expression trees as children are processed before parents.

Recursive - DFS (Depth-First Search)

Problem: Perform depth-first search on a graph to visit all reachable vertices.
Pseudocode:

Algorithm 11: Depth-First Search (Recursive)

```
Input  : Graph G (adjacency list), start vertex v, visited array
Output: DFS traversal order

Function DFS(G, v, visited):
    Mark visited[v] ← true;
    Visit vertex v (e.g., print v);
    foreach neighbor u of v in G do
        if visited[u] = false then
            DFS(G, u, visited);
        end if
    end foreach
end
```

Time Complexity: $\mathcal{O}(V + E)$ where V is vertices and E is edges
Explanation: Each vertex is visited once ($\mathcal{O}(V)$) and each edge is explored once ($\mathcal{O}(E)$). For adjacency list representation, checking neighbors of a vertex takes time proportional to its degree, summing to $\mathcal{O}(E)$ overall.

Recursive - Permutation Generation

Problem: Generate all permutations of n distinct elements using recursion in Lexicographic Order.
Pseudocode:

Algorithm 12: Permutations in Lexicographic Order

```
Input  : Array A, current position pos, boolean array used, result array perm
Output: All permutations in lexicographic order

Function PermuteLexico(A, pos, used, perm):
    if pos = length of A then
        Print perm;
        return;
    end if
    for i ← 0 to length of A − 1 do
        if used[i] = false then
            perm[pos] ← A[i];
            used[i] ← true;
            PermuteLexico(A, pos + 1, used, perm);
            used[i] ← false
        end if
    end for
end
```

Time Complexity: $\mathcal{O}(n \cdot n!)$
Explanation: There are $n!$ permutations to generate. For each permutation, we need $\mathcal{O}(n)$ time to print it. The recursion explores all possible arrangements: at the first position we have n choices, at the second position $n - 1$ choices, and so on. The total number of operations is $n \times (n - 1) \times \cdots \times 1 = n!$, and printing each takes $\mathcal{O}(n)$, giving overall complexity of $\mathcal{O}(n \cdot n!)$. The backtracking ensures we explore all permutations systematically.

*Recursive - Combination Generation

Problem: Generate all combinations of choosing k elements from n distinct elements using recursion in Lexicographic Order.
Pseudocode:

Algorithm 13: Generate All Combinations (Recursive)

Input : Array A of n elements, choose k elements, start index *start*, current combination *comb*, position *pos*
Output: All $\binom{n}{k}$ combinations

```
Function Combine(A, k, start, comb, pos):
    if pos = k then
        Print combination comb[0..k − 1];
        return;
    end if
    for i ← start to length of A − 1 do
        comb[pos] ← A[i];
        Combine(A, k, i + 1, comb, pos + 1);
    end for
end
```

Time Complexity: $\mathcal{O}(k \cdot \binom{n}{k})$
Explanation: There are $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ combinations to generate. For each combination, we need $\mathcal{O}(k)$ time to print it. The recursion explores all valid selections by choosing elements from left to right without repetition.

*Recursive - The partition problem

Problem: The linear partition problem involves partitioning a sequence of n positive numbers $a_1, a_2, \ldots, a_n$ into k consecutive ranges to minimize the maximum sum among all ranges. Given a sequence and a number of partitions k, we want to place $k-1$ dividers to create k contiguous subsequences such that the largest sum of any subsequence is minimized. We assume that:

$$S(i,j) = \sum_{m=i}^{j} a_m$$

Then, the recursive algorithm is:

- $n = k$ or $k = 1$:
$$M(n,k) = S(1,n)$$
- $n \geq k$:
$$M(n,k) = \min_{i=k-1}^{n-1} [\max(M(i,k-1), S(i+1,n))]$$

The idea is to try all possible positions i for the last divider. The last partition contains elements from $i+1$ to n with sum $S(i+1,n)$, and the remaining i elements must be partitioned into $k-1$ ranges with cost $M(i,k-1)$. We take the maximum of these two costs and minimize over all possible positions i. Implement the algorithm to compute $M(n,k)$.

Method 1: Recursive Implementation
Pseudocode:

Algorithm 14: Linear Partition (Recursive)

Input : Array $A[1..n]$ of positive numbers, integer k number of partitions
Output: Minimum maximum sum $M(n, k)$

```
Function ComputeSum(A, i, j):
    sum ← 0;
    for m ← i to j do
        sum ← sum + A[m];
    end for
    return sum;
end
Function Partition(A, n, k):
    if k = 1 or n = k then
        return ComputeSum(A, 1, n);
    end if
    minCost ← ∞;
    for i ← k − 1 to n − 1 do
        leftCost ← Partition(A, i, k − 1);
        rightCost ← ComputeSum(A, i + 1, n);
        maxCost ← max(leftCost, rightCost);
        minCost ← min(minCost, maxCost);
    end for
    return minCost;
end
```

Time Complexity: $\mathcal{O}(k \cdot n^k)$ without memoization
Explanation: For each recursive call, we try all possible positions for the last partition ($\mathcal{O}(n)$ choices). This leads to overlapping subproblems being recomputed multiple times, resulting in exponential complexity. The recursion tree has depth k with branching factor approximately n, giving $\mathcal{O}(n^k)$ time. Computing sums adds another factor of k.

Method 2: Iterating
Pseudocode:

Algorithm 15: Linear Partition

Input : Array $A[1..n]$ of positive numbers, integer k number of partitions
Output: Minimum maximum sum $M(n, k)$

```
Function Partition(A, n, k):
    prefixSum[0] ← 0;
    for i ← 1 to n do
        prefixSum[i] ← prefixSum[i − 1] + A[i];
    end for
    for i ← 1 to n do
        M[i][1] ← prefixSum[i]
    end for
    for j ← 2 to k do
        for i ← j to n do
            M[i][j] ← ∞;
            for x ← j − 1 to i − 1 do
                cost ← max(M[x][j − 1], prefixSum[i] − prefixSum[x]);
                M[i][j] ← min(M[i][j], cost);
            end for
        end for
    end for
    return M[n][k];
end
```

Time Complexity: $\mathcal{O}(n^2 \cdot k)$
Explanation: The table has $\mathcal{O}(nk)$ entries. For each entry $M[i][j]$, we try $\mathcal{O}(n)$ possible positions for the last partition. Using prefix sums, we compute range sums in $\mathcal{O}(1)$ time. The total complexity is $\mathcal{O}(n^2k)$. This approach avoids recomputing overlapping subproblems, making it much more efficient than the naive recursive solution.